"Success!" Bill exclaims. "If we schedule this program to run every few minutes, we have a functional first version of RedFlag. Let's show it to the boss."

Sharing Among Events

Your boss is amused by the simplicity of the RedFlag DSL, but she's not completely convinced. "The people who write the events will want to share data among events," she observes. "Can I do this with your DSL? For example, can two separate events access the same variable?" she asks the two of you.

"Of course they can," Bill replies. "We have a *Flat Scope (83)*." To prove that, he whips up a new events file:

blocks/redflag_2/events.rb

```
def monthly_sales
  110   # TODO: read the real number from the database
end

target_sales = 100

event "monthly sales are suspiciously high" do
  monthly_sales > target_sales
end

event "monthly sales are abysmally low" do
  monthly_sales < target_sales
end
```

The two events in this file share a method and a local variable. You run redflag.rb, and it prints what you expected:

```
ALERT: monthly sales are suspiciously high
```

"Okay, this works," the boss concedes. "But I don't like the idea of variables and methods like monthly_sales and target_sales cluttering the top-level scope. Let me show you what I'd like the DSL to look like instead," she says. Without further ado, the boss grabs the keyboard and starts churning out code like nobody's business.

Quiz: A Better DSL

Where you're unexpectedly left alone to develop a new version of the RedFlag DSL.

Your boss wants you to add a setup instruction to the RedFlag DSL, as shown in the following code.